

- SGLang 的 RadixAttention / RadixCache 解决什么问题？

与现有系统在生成请求结束后丢弃 KV 缓存的做法不同，RadixAttention 将提示词和生成结果的缓存保留在一个基数树中，从而实现高效的前缀搜索、复用、插入及淘汰。

- SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

Page-sized KV cache 对应 pipeline 中的 KV Cache 存储与管理部分。它将 KV 按页组织和分配，Prefill 写入输入 token 的 KV，Decode 读取历史 KV 并追加新生成 token 的 KV。

Prefix reuse 对应请求调度与 Prefill 衔接处的前缀匹配和复用。RadixCache 查找已缓存的共同前缀，命中部分直接复用已有 KV，Prefill 主要计算未缓存的输入部分，从而减少重复计算。

- vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention / paged KV 机制有什么联系与差异？

vLLM 的 PagedAttention 主要解决 KV Cache 连续分配造成的显存浪费，包括空间预留、内部碎片、外部碎片，以及多序列间重复存储 KV 的问题。它将 KV Cache 划分为固定大小的块，通过块表将逻辑块映射到不连续的物理块，使注意力计算能够访问这些块；配合按需分配、引用计数和写时复制，提高显存利用率并支持 KV 共享。

它与 SGLang 的 paged KV 机制都采用非连续的分页存储，属于 KV Cache 的存储与管理层面。RadixAttention 则侧重利用基数树自动匹配、保留和复用不同请求的共同前缀 KV，并通过缓存淘汰和缓存感知调度减少重复 Prefill 计算。因此，分页存储与前缀复用可以结合使用。

两者并非完全独立：vLLM 论文也支持共享前缀，但其示例由服务提供者预先缓存指定前缀；SGLang 论文进一步强调对动态前缀及多次调用的自动匹配和复用。以上比较基于两篇论文中的设计，不代表后续版本的全部功能。依据 vLLM 论文第 3、4 节及 SGLang 论文第 3 节。